Antigravity Task Prompts — Payroll FastAPI Backend

HOW TO USE:

1. First, save antigravity-workspace-rules.md as a Workspace Rule

(Rules panel -> + Workspace) in your project folder. Do this once.

2. Then go to Agent Manager -> New Task, and paste TASK 1 below.

3. Antigravity will produce a plan Artifact first. Review it, leave

comments on it if anything's off, then approve.

4. Once it finishes and shows you the result (with browser/terminal

verification), review the diff, then start a NEW TASK with TASK 2.

5. Repeat through TASK 6. Don't paste multiple tasks into one request —

each one is sized to be a single agent run with its own plan +

verification cycle.

═══

═══ TASK 1 — Scaffold + database models + migrations

═══

═══

Set up the project scaffold exactly as described in the workspace rules folder structure. Then implement all SQLAlchemy models listed in the schema section of the rules, with proper relationships (use back_populates, not backref, for explicit two-way navigation).

Initialize Alembic, generate the first migration, and show me the migration file as an artifact before applying it. Then apply it against a local Postgres instance (assume DATABASE_URL will be in .env — create .env.example with a placeholder).

Do not write any routers, services, or auth yet — just the schema and scaffold. Produce a plan artifact first showing the file list and model field mapping before you start writing, so I can confirm it matches the schema in the rules exactly.

═══

═══ TASK 2 — Authentication: register, login, JWT, role dependencies

═══

═══

Implement authentication end to end:

- app/utils/security.py: password hashing (passlib bcrypt), create_access_token() and decode_access_token() using python-jose, with expiry read from config.
- app/dependencies/auth.py: get_current_user dependency (decodes Bearer token, re-fetches the user from DB to confirm they still exist) and a require_role(*roles) dependency factory.
- app/routers/auth.py: POST /api/auth/register (public) POST /api/auth/login (public, returns access_token + user; use the SAME generic error for "no such user" and "wrong password" — don't leak which emails are registered) GET /api/auth/me (protected, returns user + employee profile if any)

After implementing, use the terminal subagent to start the server and the browser subagent to actually call /register then /login with a test account, and show me the real response — don't just tell me it works, demonstrate it.

═══

═══ TASK 3 — Employee CRUD + salary components

═══

═══

Implement app/routers/employees.py with Pydantic schemas in app/schemas/employee.py:

GET /api/employees — ADMIN only, paginated, filterable by department/status/search GET /api/employees/{id} — ADMIN, or EMPLOYEE viewing only their own record (enforce this at the query level — an employee requesting someone else's id must get 403, not just a hidden frontend button) POST /api/employees — ADMIN only, links to existing user_id PATCH /api/employees/{id} — ADMIN only, partial update POST /api/employees/{id}/salary-components — ADMIN only

Validate: base_salary > 0, ifsc_code matches ^[A-Z]{4}0[A-Z0-9]{6}$, bank_account alphanumeric 9-18 chars.

Wire in audit logging on the POST and PATCH endpoints per the rules (who changed what, when).

Verify with the browser subagent: register a second user, create an employee profile for them as admin, then try fetching that employee's record while authenticated as a

DIFFERENT employee account and confirm you get a 403. Show me that specific test result.

═══

═══ TASK 4 — Payroll engine (the most important task — read carefully)

═══

═══

This is the core of the entire system. Before writing any code, produce a plan artifact that walks through exactly how you will implement per-employee transaction isolation, and wait for my approval on that plan before implementing.

Implement app/services/payroll_service.py:

1. calculate_net_salary(employee, salary_components) -> dict Pure function, no DB access. gross = base_salary + sum(allowance amounts, converting percentage-based ones to base_salary * pct/100), deductions = same logic for DEDUCTION type, net = max(0, gross - deductions). All decimal, rounded to 2 places.
2. trigger_payroll_run(db, month, year, admin_user_id)
   - Reject if a COMPLETED or PARTIAL run already exists for (month, year)
   - Fetch all ACTIVE employees with currently-effective salary components
   - Create PayrollRun with status=PROCESSING
   - Schedule process_transactions as a FastAPI BackgroundTask — the endpoint must return immediately, not block until processing finishes
   - Return run id + status=PROCESSING right away
3. process_transactions(run_id, employees) — runs as the background task
   - Process employees ONE AT A TIME, each in its OWN database session/transaction — never one transaction for the whole batch
   - On success: write PayrollTransaction(status=SUCCESS), create a PAYROLL_CREDITED notification, commit
   - On any exception for one employee: roll back only that employee's work, write a fresh PayrollTransaction(status=FAILED, failure_reason=<error>), and continue to the next employee — never let one failure stop the batch
   - After the loop: set PayrollRun.status to COMPLETED (no failures), FAILED (no successes), or PARTIAL (mixed), with completed_at set
4. retry_failed_transactions(db, run_id) — deletes FAILED transaction rows for that run and reprocesses only those employees, without touching anyone who already succeeded

Then app/routers/payroll.py: POST /api/payroll/trigger — ADMIN, body {month, year} GET /api/payroll/runs — ADMIN, paginated GET /api/payroll/runs/{id} — ADMIN, full detail incl. transactions POST /api/payroll/runs/{id}/retry — ADMIN GET /api/payroll/my-transactions — EMPLOYEE, own history only

VERIFICATION (do this yourself, show me real output, this is not optional): write a test that seeds 3 employees, monkeypatches or forces one employee's processing to throw an exception, runs the payroll trigger, and asserts: 2 transactions are SUCCESS, 1 is FAILED with a reason, and the run status is PARTIAL — not FAILED, not COMPLETED. Run this test and paste me the actual pass/fail output.

Also add a code comment block at the top of the file explaining why per-employee transactions were chosen over one big transaction — I need to explain this tradeoff in an interview.

TASK 5 — Audit log, payslips, analytics, notifications

Implement three remaining features:

A) Audit logging (app/services/audit_service.py): a log_action() function, wired as a dependency or decorator (pick one, tell me which and why) into employee update and payroll trigger routes. Add GET /api/reports/audit-log (ADMIN, paginated, filterable).

B) Payslips + analytics in app/routers/reports.py: GET /api/reports/payslip/{transaction_id} — ADMIN any, EMPLOYEE only their own. Mask bank account to last 4 digits. Return itemized earnings/deductions and a formatted payslip number like PAY-{year}-{month:02d}-{first 8 chars of transaction id}. GET /api/reports/analytics — ADMIN only: monthly payout totals for a year, headcount/avg salary/total cost by department, and overall stats (active employees, total payouts this year, failed transaction count).

C) Notifications (app/services/notification_service.py + app/routers/notifications.py): create_notification, get_user_notifications (with unread_count), mark_as_read, broadcast_to_all_employees. GET /api/notifications, PATCH /api/notifications/read, POST /api/notifications/broadcast (ADMIN only). Confirm create_notification is actually being called from process_transactions in the payroll service — wire it in if it isn't already connected.

TASK 6 — Seed data, full pytest suite, end-to-end verification

---

1. Write a seed script: 1 admin, 5 employees (varied departments, salaries 50k-95k), each with an HRA allowance (40% of base) and a PF deduction (12% of base) salary component.
2. Write tests/test_payroll_service.py covering calculate_net_salary: base only, fixed allowance, percentage deduction, mixed allowances+deductions, and the net-never-negative edge case.
3. Run the full pytest suite and show me real output, not a summary.

4. Use the terminal subagent to start the server, run the seed script, then use the browser subagent to: log in as admin, trigger payroll for the current month, poll the run until COMPLETED, fetch one employee's payslip, and show me the actual numbers returned — confirm they match what calculate_net_salary would produce by hand for that employee's salary and components.
5. Generate a README.md documenting: how to run locally, all endpoints with example curl commands, and a one-paragraph explanation of the fault-tolerant payroll design — written so I can hand it to an interviewer.